

SafePay — Product Requirements Document (PRD)

Risk-Adaptive Payment Protection Platform for Indian Bank Transfers (NEFT / RTGS / IMPS / UPI)

Document type: Master PRD for engineering, design, and agentic build tools (Cursor / Codex / Figma AI)

Platform: Progressive Web App (PWA) — customer + corporate + admin surfaces

Version: 1.0 · **Status:** Ready for build

0. How to Use This Document

This PRD is written to be handed directly to a developer or an agentic coding/design tool and built **exactly** as specified — screens, states, data, and APIs are fully enumerated so nothing is left to inference. It intentionally contains **no full implementation code**. Where logic is genuinely non-trivial (risk scoring, state transition guard), a short pseudocode/template is given — never a complete class or module. Everything else (CRUD, forms, standard REST controllers) is left to the build tool's normal patterns.

Source materials consolidated in this PRD: Group Project Brief, Research Foundation (payment-rail risk landscape + ledger schema patterns), Project Blueprint (data model, APIs, tech mapping), and the full SafePay Practical Solution + User/Admin Flow document. Every case, rule, table, and edge case across all four source documents has been folded in below — nothing from the source material has been dropped, only reorganized and made build-ready.

1. Positioning & Non-Negotiable Framing

SafePay is a simulated, risk-adaptive, pre-settlement transaction-control layer. It does not intercept, pause, or reverse a live NEFT/RTGS/IMPS/UPI transaction on real payment rails. It is a policy/orchestration layer sitting between "payment authorized" and "payment settled" inside its **own simulated ledger**.

Tagline: *Fast when safe. Careful when necessary.*

What SafePay must never claim (hard constraint on all copy, marketing screens, and onboarding text)

- "We invented fraud detection."
- "Banks cannot detect these transactions."
- "SafePay can reverse any UPI/IMPS payment."
- "Our timer works on live UPI."
- "SafePay completely prevents online fraud."

Preferred product-positioning line (use verbatim in About/Help screens): *"SafePay is a simulated risk-adaptive pre-settlement transaction-control layer."*

Why this matters for the build

Every screen that mentions "hold," "protect," "undo," or "settlement" must use **simulated-safe language** — e.g., "Simulated Settlement," "SafePay Protection Window" — never implying a real bank transfer was altered. A persistent, non-intrusive badge/label pattern (e.g., a small "Simulation" tag near balance and settlement confirmations) is a required UI element, not optional polish.

2. Problem Statement (Dual POV: Banker + Customer)

2.1 Customer-side problem

Instant rails (UPI/IMPS) and even batch/RTGS rails are optimized for speed and finality, not reversibility:

- **NEFT/RTGS/IMPS:** routing is account-number-based; a wrong digit sends money to a real, valid, unintended account. Once settled, recovery depends entirely on the receiving bank persuading the receiver to voluntarily return funds — there is no legal compulsion to debit them back.
- **IMPS:** inherits this problem with **zero in-transit window** — no batch delay to catch it.
- **UPI:** the highest-fraud-volume rail because the fraud mechanism is **social engineering, not hacking** — the victim actively authorizes the payment (approves a collect request, shares an OTP, installs a screen-mirroring app under a scammer's instruction).
- Common human triggers: wrong beneficiary selection (similar name), fat-fingered amount, duplicate/retry payment after a perceived failure, payment to a beneficiary added minutes ago, payment made under scam-induced urgency.
- Regulatory backstop today (RBI unauthorized-transaction liability rules) rewards *speed of reporting after the fact* — it does nothing to stop the mistake *before* it happens.

2.2 Banker/bank-ops-side problem

- Every wrong-credit or fraud case becomes a **manual, cross-bank, goodwill-dependent recovery** process — expensive in staff time, slow, and reputationally risky, with no guaranteed resolution.
- Existing controls (auth, beneficiary look-up, fraud monitoring, post-transaction disputes) are all **reactive** — there is no layer that takes the risk signals these systems already produce and turns them into one real-time, explainable, configurable **decision**.
- Uniform "cooling-off" rules (e.g., a fixed new-beneficiary hold) apply the same friction to a ₹500 payment and a ₹5,00,000 payment — either under-protecting the large payment or needlessly annoying the small one.
- Corporate/business customers have no lightweight way to add a second set of eyes to unusually large payments without imposing full manual approval on *every* payment.

2.3 SafePay's answer

Turn transaction risk signals into a single configurable orchestration decision:

```
Payment → Risk Signals → Risk Score → Policy Decision → Release /  
Protect / Hold
```

This must be positioned as an **orchestration/decision layer**, not a novel fraud-detection invention.

3. Goals, Non-Goals, and Design Principles

3.1 Goals

1. Give retail customers a real chance to catch and undo mistakes before irreversible (simulated) settlement.
2. Give corporate customers a lightweight Maker-Checker control for high-value payments.
3. Give bank ops/admin a transparent, configurable, auditable policy engine — not a black box.
4. Keep low-risk payments **instant** — friction must be earned by risk, never applied uniformly.
5. Make every risk decision **explainable** in plain language to the end user and in full technical detail to the admin/auditor.

3.2 Non-Goals (explicitly out of scope for this build)

- Real integration with NPCI/UPI/IMPS/NEFT/RTGS rails.
- Production-grade KYC/AML or money-mule detection on the receiving end.
- ML/AI as the primary or sole risk-decision authority (AI is optional, advisory-only, and gated — see §11.5).
- Real biometric authentication, real SMS/email delivery, real escrow.
- Multi-bank settlement or ledger reconciliation against real bank statements (schema is designed to support it later — see §14.6 — but it is not built in this phase).

3.3 Core design principles (apply to every screen and every service)

Principle	What it means in practice
Adaptive friction, not uniform friction	Never show a timer, OTP, or warning for a LOW-risk payment.
Server is the only source of truth for time	The frontend never owns a countdown; it only renders <code>protected_until - now()</code> and re-syncs.
Explainability over black-box	Every protection/hold screen must show <i>why</i> in plain language ("new beneficiary + amount 9× your average"), never an opaque "flagged" message.
Non-accusatory language	Never imply the <i>beneficiary</i> is fraudulent — <u>the warning is about the pattern</u> , not an accusation against a person.
Idempotency everywhere	Every state-changing request (<code>create</code> , <code>authorize</code> , <code>cancel</code> , <code>release</code> , <code>approve</code>) requires an idempotency key.
Append-only audit	No UI, anywhere, for any role, may edit or delete an audit record.
Simulation transparency	Every settlement/balance screen visibly signals this is a simulated environment.

4. Personas

Persona	Role	Primary goals	Key screens
Riya — Retail Customer	First-time/occasional digital payer	Send money fast when it's routine; get a real chance to undo a mistake	Dashboard, New Payment, Protection Window, History

Persona	Role	Primary goals	Key screens
Arjun — Corporate Maker	Business ops executive	Create vendor/payroll payments quickly	Corporate Dashboard, New Payment, Approval Status
Meera — Corporate Checker	Finance manager / approver	Review and approve/reject high-value payments with full context	Approval Queue, Transaction Detail
Karan — Risk Officer (Admin)	Bank ops risk reviewer	Review VERY_HIGH holds, adjust policy, monitor system health	Risk Review Queue, Policy Config, Ops Dashboard
System Admin	Platform administrator	Manage users/roles, lock accounts, monitor scheduler/AI health	User Management, System Health
Auditor	Compliance/read-only reviewer	Inspect immutable history of everything	Audit Viewer (read-only, all modules)

5. Scope Tiering (MVP → Phase 2 → Future)

5.1 MVP (must work end-to-end for the first demo/release)

1. Register/Login (JWT), role-based routing.
2. Add/manage beneficiaries with cooling-off metadata.
3. Create payment → simulated authentication (PIN/OTP mock) → risk evaluation.
4. Four risk tiers (LOW/MEDIUM/HIGH/VERY_HIGH) mapped to instant settlement / 10s protection / 60s protection / hard hold + OTP.
5. Full transaction state machine with server-authoritative expiry (scheduled job).
6. Cancel/Undo during protection window.
7. OTP step-up for VERY_HIGH with attempt/expiry handling.
8. Transaction history with plain-language risk explanation.
9. Basic Admin Dashboard + Risk Review Queue + Audit Viewer.
10. Duplicate-payment prevention via idempotency key.
11. In-app notifications for every state transition.

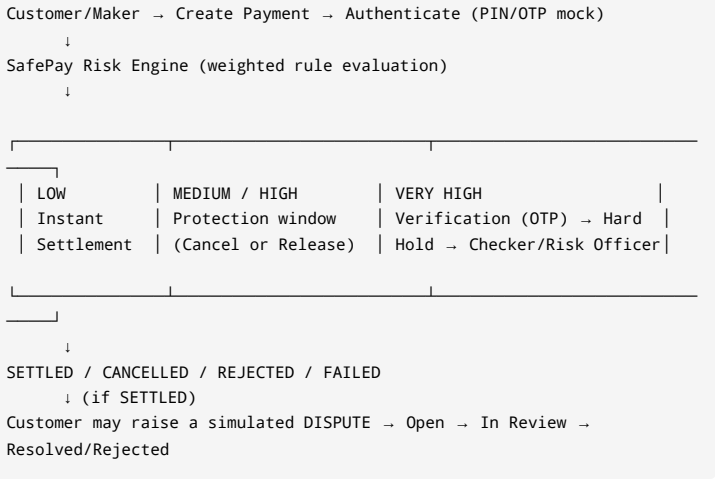
5.2 Phase 2 (build once MVP is stable)

- Corporate Maker-Checker mode (roles, approval queue, approval limits, maker=checker enforcement).
- User-configurable safety preferences (protect-above-amount, always-protect-new-beneficiary, custom window length).
- Dispute workflow (post-settlement).
- Admin-configurable risk weights/thresholds/timer durations with versioning.
- Reporting views (transaction dashboard, risk summary, pending approvals, customer protection analytics).
- PWA installability, offline shell, push notifications.

5.3 Future scope (explicitly deferred – state this in any stakeholder-facing document)

- AI/ML-based anomaly detection and dynamic risk scoring (advisory-only, see §11.5).
- Multi-level corporate approvals, document verification, separation-of-duties tooling.
- Escrow/conditional-release payment type.
- Real bank-statement reconciliation job against `ledger_entry` (schema ready, job not built).
- Real NPCI/RBI sandbox integration.

6. End-to-End Flow (Canonical)



6.1 Retail mode example

High-risk payment → Protected 60s → Customer chooses Cancel or lets it Release → Settled.

6.2 Corporate mode example

Maker creates ₹25,00,000 vendor payment → Hold (VERY_HIGH or policy-forced) → routed to Checker queue instead of a timer → Checker Approves/Rejects with remarks → Settled or Cancelled.

7. Risk Engine Specification

7.1 Philosophy

MVP uses a transparent, deterministic, rule-based engine only. AI/ML is explicitly *not* the primary feature in v1 — see §11.5 for the gated, advisory-only future AI adapter.

7.2 Risk signals and weights (MVP configuration — stored in DB, admin-editable in Phase 2)

Risk Signal	Weight
New beneficiary (< 24h old)	+25 to +30
High absolute amount	+25
First transaction to this beneficiary	+15 to +20
Amount exceeds user-defined limit	+35
Amount > 3× user's historical average	+20
New/unrecognized device	+20
Unusual/unfamiliar location	+15
Unusual time of day	+5
Transaction velocity (>5 payments in 10 minutes)	+20
Corporate amount exceeds approver's limit	+40
Known, trusted beneficiary + normal pattern	-15 (trust discount)

Weights are project configuration values, not claims about real banking rules. They must live in a `RISK_FACTORS` table, never hardcoded, so the Admin Policy screen (§13) can edit them without a redeploy.

7.3 Tiers, thresholds, and actions

Risk Score	Tier	Action	Protection Window
0–19 / 0–30	LOW	Instant settlement	0s
20–39 / 31–60	MEDIUM	Short protection, Cancel/Release available	10s
40–69 / 61–85	HIGH	Longer protection, stronger warning	60s
70+ / 86+	VERY_HIGH	OTP step-up → hard hold → manual/Checker review	0s timer, hold until resolved

Two threshold sets appear across the source material (project brief vs. practical-solution doc). **Default to the practical-solution doc's bands (0–19/20–39/40–69/70+)** as the shipped default, but the boundary values must be admin-configurable, not hardcoded, precisely because both documents proposed different numbers — this is a policy knob, not a fixed law.

7.4 Explainability requirement

Every assessment must persist a `risk_signal_log` / `TRANSACTION_RISK_FACTORS` row per contributing signal (signal name, raw value, weight, contribution to score) so that:

- The **customer-facing** explanation is a plain-language sentence built from the top 1–2 contributing signals (e.g., *"Protected because: new beneficiary + amount 9× your average"*).
- The **admin/auditor-facing** explanation shows the full signal breakdown, weights, and running total.

7.5 Risk scoring — reference pseudocode (illustrative only, not a full implementation)

```
score = 0
for rule in activeRiskRules:
    signal = rule.evaluate(context)
    score += signal.weight
    explanation.add(signal.reason)

tier = classify(score, currentPolicyThresholds) // LOW / MEDIUM / HIGH / VERY_HIGH
```

```
return immutable RiskAssessment(score, tier, protectionWindow(tier),
requiresVerification(tier), explanation)
```

7.6 Risk-engine failure handling

- If a rule fails to evaluate (data missing, service error), the engine must **default to the more conservative (safer) action** for that signal – never silently treat a failure as LOW risk.
- If the entire risk service is unavailable, deterministic Java rules must still run; AI unavailability specifically must **never** cause an automatic LOW classification (see §11.5.4).

8. Transaction State Machine

8.1 Canonical states

```

CREATED → AUTHORIZED → RISK_ASSESSED
  └─ LOW → RELEASED → SETTLED
  └─ MEDIUM / HIGH → PROTECTED → CANCELLED | RELEASED → SETTLED
  └─ VERY_HIGH → VERIFICATION_REQUIRED → PROTECTED (hard
hold) → PENDING_APPROVAL → APPROVED/REJECTED → SETTLED | CANCELLED
SETTLED → DISPUTED → UNDER_REVIEW → RESOLVED | REJECTED
(any eligible state) → FAILED

```

8.2 Transition table

Current State	Trigger	Validation	Next State
CREATED	User authorizes	Valid beneficiary, amount, user authenticated	AUTHORIZED
AUTHORIZED	Risk = LOW	Rule engine completes	RELEASED
AUTHORIZED	Risk = MEDIUM/HIGH	Timer calculated	PROTECTED
AUTHORIZED	Risk = VERY_HIGH	Step-up required	VERIFICATION_RE
PROTECTED	User cancels	<code>now < protection_expires_at</code>	CANCELLED

Current State	Trigger	Validation	Next State
PROTECTED	Timer expires	Not cancelled AND optimistic lock succeeds	RELEASED
VERIFICATION_REQUIRED	OTP succeeds	Valid, non-expired OTP	PROTECTED or RELEASED
VERIFICATION_REQUIRED	OTP fails/max attempts	Policy evaluation	CANCELLED or F A
RELEASED	Settlement runs	Not previously settled	SETTLED
SETTLED	Refund/dispute requested	New workflow only	remains SETTLED
CANCELLED	Any release/settle request	Invalid – reject	remains CANCELLED

Explicitly invalid transitions (must be guarded in code): SETTLED → CANCELLED ,
CANCELLED → RELEASED , CREATED → SETTLED (direct).

8.3 Concurrency & failure edge cases (must all be handled)

Scenario	Required behavior
Cancel exactly at expiry	DB update requires <code>state='PROTECTED' AND protection_expires_at > now()</code> ; only one competing write wins. UI must reflect whichever the DB actually committed – never show "cancelled" unless the DB confirms it.
Duplicate requests (network retry)	Idempotency-Key header required on every state-changing call; request hash + response cached per key.
Two release requests	Optimistic locking via a <code>version</code> column + conditional update.
Server/scheduler restart	Protection expiry lives in the DB, not memory; on restart the scheduler re-queries all expired <code>PROTECTED</code> rows and resumes.
Multiple scheduler instances	Conditional/optimistic-locked update ensures only one instance wins the release.
AI service unavailable	Deterministic rules run; conservative protection policy applied; never falls back to LOW.

Scenario	Required behavior
Network failure after authorization	Client polls <code>GET /transactions/{id}</code> using the same idempotency key/client-generated ID to recover state.
Cancel attempted after settlement	Reject; offer simulated refund/dispute request instead.
Maker = Checker (same person)	Enforced at query <i>and</i> service layer: <code>maker_user_id != checker_user_id</code> .
OTP expires mid-flow	User may request a new OTP within policy-defined limits.
Settlement itself fails	Transaction must never be marked SETTLED on a failed attempt; log failure, retry using the same idempotency key (prevents double-debit), or move to FAILED for admin investigation.

8.4 Locking strategy (required, not optional)

1. `@Transactional` boundaries around every state-changing DB operation.
2. Optimistic locking via a `version` column on `TRANSACTIONS`.
3. Conditional SQL updates, e.g. `WHERE state = 'PROTECTED' AND version = :version`.
4. Optional short-lived in-process lock keyed by transaction ID for same-instance contention — **never** relied upon as the sole safeguard, since multiple app instances may run.

9. Data Model

9.1 Two-layer design (business-status + immutable ledger)

Per the research foundation, SafePay's data layer must **separate the mutable business-status view from an immutable ledger**, mirroring how real cloud-native banking cores work:

- `TRANSACTIONS` (a.k.a. `PAYMENT_TRANSACTIONS`) — the account-facing, mutable-status table used by UI and business logic (this is the "Transaction" entity most of the flows above reference).
- `LEDGER_ENTRY` — append-only, immutable, double-entry postings created only when a transaction reaches SETTLED.
- `TRANSACTION_EXCEPTION` — every failure/bounce/mismatch lands here instead of silently disappearing.

This two-layer design is what distinguishes the schema from a basic CRUD app.

9.2 Core tables

Table	Purpose
ROLES / USER_ROLES	Role master data (CUSTOMER, MAKER, CHECKER, RISK_OFFICER, SYSTEM_ADMIN, AUDITOR)
USERS (APP_USER)	Customers, Makers, Checkers, Admins
ACCOUNTS	Simulated bank accounts and balances, rolling average spend
BENEFICIARIES	Saved recipients, trust metadata
TRANSACTIONS	Main payment + state table
RISK_FACTORS	Configurable risk rules/weights
TRANSACTION_RISK_FACTORS (RISK_SIGNAL_LOG)	Why a transaction received its score
PROTECTION_RULES / PROTECTION_POLICY	Maps risk tier → action/protection duration
USER_SAFETY_SETTINGS / USER_PREFERENCE	Customer safety preferences
APPROVALS (MAKER_CHECKER_APPROVAL)	Maker-Checker records
DISPUTES	Post-settlement dispute workflow
AUDIT_LOG	Append-only history of all state changes
OTP_CHALLENGES	Step-up verification challenges, expiry, attempt count
DEVICE_PROFILES	Known/trusted device fingerprints per user
IDEMPOTENCY_KEYS	Cached request/response per idempotency key
NOTIFICATIONS	In-app + simulated email/SMS log
LEDGER_ENTRY	Immutable double-entry postings (see §9.3)

Table	Purpose
TRANSACTION_EXCEPTION	Failure/reconciliation records (see §9.4)

9.3 LEDGER_ENTRY (append-only, immutable)

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	VARCHAR2(64)	SafePay's internal reference
external_reference	VARCHAR2(64)	Simulated UTR (NEFT/RTGS) or RRN (IMPS/UPI)
source_system	VARCHAR2(30)	e.g. SAFEPAY , NEFT , UPI
idempotency_key	VARCHAR2(64)	Unique with source_system – prevents double posting
account_id	NUMBER (FK)	
entry_type	VARCHAR2(10)	DEBIT / CREDIT
amount	NUMBER(18,2)	Never FLOAT
currency	VARCHAR2(3)	INR
status	VARCHAR2(20)	POSTED / REVERSED
reversal_of_entry_id	NUMBER (nullable, FK to self)	Points to the entry being reversed
created_at	TIMESTAMP	Write-once
effective_at	TIMESTAMP	When the movement is considered to have happened

Constraint: UNIQUE (source_system, idempotency_key) – the single constraint that prevents duplicate postings from retries.

Rule: never edit or delete a posted row. If a transaction is wrong, post a new reversal entry.

9.4 TRANSACTION_EXCEPTION

Column	Type	Notes
id	NUMBER (PK)	
transaction_id	VARCHAR2(64) (FK)	Links to the original attempted transaction
failure_stage	VARCHAR2(30)	AUTHORIZATION , RAIL_SUBMISSION , SETTLEMENT_CONFIRMATION
error_code	VARCHAR2(20)	INVALID_IFSC , ACCOUNT_FROZEN , TIMEOUT , INSUFFICIENT_FUNDS
error_message	VARCHAR2(255)	Human-readable detail
debited_not_credited	CHAR(1)	Y/N – money left sender but didn't arrive
retry_count	NUMBER	
resolution_status	VARCHAR2(20)	PENDING / RESOLVED / AUTO_REVERSED / MANUAL_REVIEW
resolved_at / created_at	TIMESTAMP	

9.5 Key TRANSACTIONS columns

transaction_id, customer_id, from_account_id, beneficiary_id, amount (NUMBER 18,2), currency, state, risk_tier, risk_score, protection_window_seconds, protection_expires_at, requires_checker (Y/N), device_id, location, remarks, version (optimistic lock), idempotency_key, created_at, authorized_at, released_at, settled_at, cancelled_at, updated_at.

Money rule (non-negotiable): every monetary column is NUMBER(18,2) / DECIMAL in the DB and BigDecimal in application code. **Never** FLOAT or double.

9.6 Relationships (summary)

```
app_user (1)—(*) account
account (1)—(*) beneficiary
account (1)—(*) transaction [from_account]
beneficiary (1)—(*) transaction
transaction (1)—(*) risk_signal_log
transaction (1)—(0..1) maker_checker_approval
transaction (1)—(0..1) dispute
app_user (1)—(1) user_preference
transaction (1)—(*) ledger_entry [on SETTLED, a balanced
```

```
debit/credit pair]
transaction (1) — (0..*) transaction_exception
```

9.7 Oracle-specific requirements

- Oracle sequences for every generated ID: SEQ_USER_ID, SEQ_ACCOUNT_ID, SEQ_BENEFICIARY_ID, SEQ_TRANSACTION_ID, SEQ_RISK_FACTOR_ID, SEQ_RULE_ID, SEQ_APPROVAL_ID, SEQ_DISPUTE_ID, SEQ_AUDIT_ID.
- `@GeneratedValue(strategy = GenerationType.SEQUENCE)` in JPA entities, mapped 1:1 to the tables above.
- Primary/foreign keys, NOT NULL / UNIQUE / CHECK constraints, and seed sample data at the DB layer, not just app-layer validation.

9.8 Reporting views (Phase 2)

View	Purpose
VW_TRANSACTION_DASHBOARD	Admin transaction monitoring
VW_RISK_SUMMARY	Risk-tier counts, amounts, and averages
VW_PENDING_APPROVALS	Maker-Checker queue
VW_CUSTOMER_PROTECTION_ANALYSIS	Customer-level safety analytics

10. API Surface

Full CRUD/auth boilerplate is intentionally not detailed here — the build tool should scaffold it in the framework's standard REST style. Endpoints below define the **contract shape**; only the two most complex ones include a short sample payload.

10.1 Auth

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
```

10.2 Accounts & Beneficiaries

```
GET    /api/accounts/{id}
GET    /api/accounts/{id}/balance
POST   /api/beneficiaries
GET    /api/beneficiaries?accountId={id}
```

```
PATCH /api/beneficiaries/{id}/status
DELETE /api/beneficiaries/{id}
```

10.3 Transactions (core engine)

```
POST /api/transactions/initiate          → runs risk engine,
returns decision + window
POST /api/transactions/{id}/authorize    → simulated OTP/PIN step
GET /api/transactions/{id}
GET /api/transactions?accountId={id}&status={status}
POST /api/transactions/{id}/verify      → submit step-up OTP
POST /api/transactions/{id}/cancel
POST /api/transactions/{id}/release     → manual early release
(policy-gated)
GET /api/transactions/{id}/risk-explanation
```

Sample — POST /api/transactions/initiate (illustrative shape only):

```
// Request (requires header: Idempotency-Key: <uuid>)
{
  "fromAccountId": 1042,
  "beneficiaryId": 88,
  "amount": 450000.00,
  "purpose": "Vendor payment",
  "deviceId": "dev_9f21",
  "location": "Pune, IN"
}
// Response
{
  "transactionId": "TXN-2026-000481",
  "state": "PROTECTED",
  "riskTier": "HIGH",
  "riskScore": 58,
  "protectionExpiresAt": "2026-09-02T10:31:06Z",
  "explanation": ["Beneficiary added 15 minutes ago", "Amount is
unusually large for this beneficiary"]
}
```

10.4 Maker-Checker (Phase 2)

```
GET /api/approvals/pending?checkerId={id}
POST /api/approvals/{id}/approve
POST /api/approvals/{id}/reject
```

10.5 Preferences (Phase 2)

```
GET /api/preferences/{userId}
PUT /api/preferences/{userId}
PATCH /api/settings/safety
```

10.6 Disputes (Phase 2)

```
POST /api/disputes
GET /api/disputes/{id}
PUT /api/disputes/{id}/status
```

10.7 Admin

```
GET /api/admin/policies
PUT /api/admin/policies/{riskLevel}
GET /api/admin/audit-logs
GET /api/admin/dashboard/stats
GET /api/audit/transactions/{id}
```

10.8 Notifications

```
GET /api/notifications?userId={id}
```

10.9 Real-time

```
WS /ws/transactions/{id}/status → pushes live countdown + state
changes (STOMP over WebSocket)
```

Rule: the WebSocket push is a display convenience only. The frontend must never treat a missed/late WS tick as a state change — it must reconcile against `GET /api/transactions/{id}` on reconnect.

11. Backend Architecture

11.1 Module boundaries

Module	Responsibility
auth	Registration, login, JWT issuance, roles
beneficiary	Add, validate, list, deactivate

Module	Responsibility
transaction	Creation, authorization, state transitions, idempotency
risk	Rules, score calculation, protection decision
verification	OTP challenge issuance and validation
notification	In-app + simulated email/SMS
audit	Append-only event logging
corporate	Maker-Checker workflow (Phase 2)
ai	Isolated, optional recommendation adapter (Future – see §11.5)

11.2 Core domain types (illustrative signatures only – not full classes)

```
enum TransactionState { CREATED, AUTHORIZED, PROTECTED,
                        VERIFICATION_REQUIRED,
                        CANCELLED, RELEASED, SETTLED, FAILED, DISPUTED
}
enum RiskTier { LOW, MEDIUM, HIGH, VERY_HIGH }

record RiskAssessment(int score, RiskTier tier, Duration
protectionWindow,
                    boolean requiresVerification, List<String>
reasons) {}

interface RiskRule { RiskSignal evaluate(RiskContext context);
}
interface RiskAssessmentService { RiskAssessment assess(RiskContext
context); }
interface TransactionStateService {
    PaymentTransaction authorize(UUID id);
    PaymentTransaction cancel(UUID id, String idempotencyKey);
    PaymentTransaction release(UUID id, String idempotencyKey);
    PaymentTransaction settle(UUID id);
}
```

11.3 Patterns to apply (mapped to concepts already chosen by the team)

Pattern/Concept	Where it applies
Strategy	Individual risk rules (amount, device, beneficiary, frequency)
State	Centralized transaction-transition service (§8)

Pattern/Concept	Where it applies
Chain of Responsibility	Sequential risk checks accumulating signals into a context
CompletableFuture	Parallel device/amount/beneficiary-history/frequency checks
Generics	<code>ApiResponse<T></code> , <code>RiskRule<T></code> , paginated results
Custom exceptions	<code>InvalidStateTransitionException</code> , <code>DuplicateTransactionException</code> , <code>VerificationFailedException</code>
Immutable value objects	<code>RiskAssessment</code> as a <code>record</code>
Scheduled tasks	Auto-release of expired <code>PROTECTED</code> transactions
Thread safety	Per-transaction short-lived lock + DB optimistic locking

11.4 Scheduler behavior (critical path)

A `@Scheduled` job periodically scans `PROTECTED` rows where `protected_until < now()` and conditionally transitions them to `RELEASED` → `SETTLED`. This is what makes the **server**, not the browser, authoritative over the countdown. On restart, the scheduler must re-query rather than rely on in-memory timers (see §8.3).

11.5 Restricted Risk Investigation Agent (Future, advisory-only — build only after MVP is stable)

This is the **only** place AI may appear in the system, and only under hard constraints:

1. **Workflow:** backend gathers permitted, masked transaction facts → agent returns a risk explanation, a confidence score, and a *recommended* action (immediate release / temporary pause / extra verification / human review) → Java validates the recommendation against fixed policy rules → the recommendation is logged to audit → **only the Java state-transition service may actually release, cancel, or settle.**
2. **Permitted data:** amount + timestamp, beneficiary age/trust indicators (masked identifier), device trust status, *coarse* location-risk status (never precise location without consent), the customer's own aggregated history, corporate approval status.
3. **Prohibited actions (hard rule):** the agent must never transfer money, settle/release/cancel a transaction, edit settings, modify audit logs, bypass state transitions, or access unrelated customers' data.

4. **Controls:** low-confidence VERY_HIGH recommendations always require human review; hard policy rules always override AI recommendations.
 5. **Failure handling:** if the AI service is unavailable, deterministic Java rules run and a conservative policy applies. **AI unavailability must never result in an automatic LOW-risk classification.**
-

12. UI/UX Specification (Progressive Web App)

12.1 Design direction

A sleek, modern, high-trust visual language that blends:

- **BHIM UPI's** simplicity: minimal steps, huge legible amount typography on payment/confirmation screens, single dominant CTA per screen, generous white space, thumb-reachable primary actions.
- **Enterprise banking (Oracle-grade) polish:** dense, well-organized data tables and dashboards for admin/risk/audit surfaces, calm neutral palette with a single strong accent for risk states, clear typographic hierarchy, restrained motion.

Note on tech-stack deviation: the source Blueprint recommends Oracle JET for the frontend. Because the explicit brief here is a **Progressive Web App**, this PRD directs the frontend to be built as a modern component-based PWA (installable, offline app-shell, responsive, push-notification capable) rather than Oracle JET — while keeping the Oracle-JET-described *interaction patterns* (module-based screens, live countdown component, Cancel/Release action pair) as the functional spec. Confirm this substitution with stakeholders before the design-system phase; the full visual system (colors, type, spacing tokens) will be supplied separately at frontend-implementation time per your note.

12.2 Global UI rules (apply everywhere)

- A **risk-tier color language** is used consistently: LOW = calm/neutral (no color alarm at all), MEDIUM = amber/caution, HIGH = orange/strong-warning, VERY_HIGH = red/hard-stop. Never use red for anything except VERY_HIGH/hold states — color must stay meaningful, not decorative.
- Every screen that shows money uses full rupee formatting with grouping (₹4,50,000), never truncated or abbreviated on a primary confirmation screen.
- A persistent small **"Simulated Environment"** tag near balance/settlement UI (see §1).
- Never expose internal rule codes, OTP values, JWTs, or another user's data anywhere in the customer UI.
- All destructive/final actions (Cancel Payment, Reject Approval) require one explicit confirmation step — no accidental taps.

- PWA requirements: installable (manifest + service worker), responsive from 360px to desktop, works offline for read-only history view (cached), push notification permission requested contextually (after first protected transaction, not on first launch).

12.3 Customer screens

Screen	Primary purpose	Key elements
Login / Register	Authenticate or create account	Email/mobile + password; generic error on bad credentials; lockout messaging after repeated failures
Dashboard (Home)	At-a-glance status	Simulated balance, recent transactions, pending protection windows surfaced prominently at the top (never buried), saved beneficiaries shortcut, notifications bell
Beneficiaries	Manage recipients	List with trust/age indicator, Add Beneficiary CTA, deactivate action
Add Beneficiary	Capture recipient	Name, bank, account/UPI ID, IFSC (conditional), nickname, optional relationship/purpose; duplicate/self-account detection inline
New Payment — Select Beneficiary	Step 1 of payment	Beneficiary name, masked account/UPI ID, beneficiary age, last-payment date, "New beneficiary" badge when applicable
New Payment — Amount & Details	Step 2 of payment	Amount (large, prominent), purpose, optional reference, source account; inline duplicate-recent-payment prompt
Payment Confirmation	Final check before authorization	"You are sending ₹X" (large), beneficiary name + masked ID, beneficiary age, applicable warnings, explicit note that risky payments may be protected; actions: Confirm & Pay / Go Back / Cancel
Protection Window	Live countdown for MEDIUM/HIGH	Beneficiary, amount, risk level badge, live server-synced countdown, plain-language risk explanation, Undo button, secondary "what happens if I do nothing" microcopy
Additional Verification (OTP)	VERY_HIGH step-up	OTP input, attempts-remaining indicator, resend (rate-limited), Cancel option

Screen	Primary purpose	Key elements
Payment Result	Outcome	Settled / Cancelled / Held / Failed state, reference number, next-step guidance
Transaction History	Search & inspect	Filter by state/date/beneficiary/amount; each row expandable to a plain-language timeline (Created → Authorized → Risk Assessed → Protected → Cancelled, with timestamps)
Transaction Detail / Timeline	Full plain-language audit view for the user	Never shows internal rule codes — only human-readable reasons
Safety Settings	User-controlled friction	Protect-above-amount, always-protect-new-beneficiary toggle, custom window length, confirm-above-amount
Notifications	Alerts	Beneficiary added, payment protected, window expiring soon, released, held for approval, dispute update
Raise a Dispute	Post-settlement only	Reason, reference to settled transaction, status tracker (Open → In Review → Resolved/Rejected)

12.4 Corporate screens (Phase 2)

Screen	Purpose
Corporate Dashboard	Maker's view of created payments and their approval status
New Corporate Payment	Same as retail New Payment flow, tagged as corporate; shows approval-limit context
Approval Queue (Checker)	List of PENDING approvals with amount, maker, risk tier, wait time
Approval Detail	Full context (identical evidence set as Admin Review, §12.5) + Approve/Reject + mandatory remarks
Corporate Audit History	Full decision timeline per payment (Future: immutable retention policy)

12.5 Admin / Risk Ops screens

Screen	Primary actions
Admin Login	Secure, separate route; supports RISK_OFFICER / SYSTEM_ADMIN / AUDITOR / CUSTOMER_SUPPORT roles
Operations Dashboard	Protected-transaction count, VERY_HIGH review queue size, failed-verification count, cancelled count, duplicate-request count, scheduler/AI health
Risk Review Queue	Transactions in VERIFICATION_REQUIRED / UNDER_REVIEW : reference, masked customer, amount, masked beneficiary, tier/score, hold duration, wait time
Transaction Review Detail	Amount vs. user's normal range, beneficiary age, prior payment count, device trust, coarse location status, frequency, verification result, full rule-engine explanation, optional AI recommendation + confidence, full audit timeline; actions: Approve for Protection / Reject-Cancel / Request Re-verification / Escalate / Add Internal Note
Transaction Monitor	Search by ID/date/customer/beneficiary/state/tier/amount; state-gated action availability (see §12.6)
User Management	Search users, view status, lock/unlock, force password reset, view registered devices, revoke sessions — never shows raw passwords, cannot silently raise limits, cannot edit past payment history
Risk Policy	Edit score boundaries, timer durations per tier, max payment limits, new-beneficiary cooling period, OTP attempt limits, review thresholds, mandatory-protection rules — every change requires a reason, is versioned, applies only to future assessments, and (recommended) can itself be Maker-Checker-gated
Audit Viewer	Read-only, append-only event history across logins, beneficiary changes, transaction lifecycle, risk-rule results, OTP events (never the OTP value itself), cancellations, scheduler releases, admin decisions, policy changes, settlement results, AI recommendations/fallbacks
System Health	Scheduler status, notification delivery status, AI service status

12.6 Admin capability by transaction state (enforced server-side, mirrored in UI as disabled/hidden actions)

Transaction State	Admin capability
CREATED	View only

Transaction State	Admin capability
AUTHORIZED	View risk-processing status
PROTECTED	View countdown; cancellation only under an explicit, logged support policy
VERIFICATION_REQUIRED	Review, request re-verification, approve or reject under policy
RELEASED	View only — cannot cancel
SETTLED	View; may start a <i>separate</i> dispute/refund workflow only
CANCELLED	View only
FAILED	View failure detail and permitted retry info

13. Roles & Permission Matrix

Capability	Customer	Maker	Checker	Risk Officer	System Admin	Auditor
Create payment	Yes	Yes	No	No	No	No
Cancel own protected payment	Yes	Yes	No	No	No	No
Approve/reject corporate payment	No	No	Yes (not own)	—	Optional	Read only
Review VERY_HIGH hold	No	No	—	Yes	Optional	Read only
Directly settle a payment	No	No	No	No	No	No
Change risk policy	No	No	No	No	Yes	Read only
Lock a user account	No	No	No	No	Yes	Read only

Capability	Customer	Maker	Checker	Risk Officer	System Admin	Auditor
View audit history	Own limited history	Own	Relevant cases	Relevant cases	Yes	Yes
Edit or delete audit records	No — nobody, ever					
Cancel a settled payment	No — for anyone					

Hard rule: `maker_user_id != checker_user_id` is enforced at both the query and service layer — a maker can never approve their own payment.

14. Notifications

Trigger points (in-app always; simulated email/SMS where noted): beneficiary added, payment protected (window started), protection window expiring soon (e.g., last 5s of a 10s/60s window), payment released, payment held for approval, checker decision made, dispute status update, OTP issued/failed, account locked.

Rule: notification delivery failure must **never block** the underlying payment action — notifications are best-effort and asynchronous to the transaction pipeline.

15. Business Rules (must be enforced server-side, not just in UI)

1. Only the transaction owner can cancel through the customer flow.
2. Cancellation is allowed only before release or settlement.
3. Every state-changing request requires an idempotency key.
4. The database is the final source of truth for transaction state — never the client, never in-memory server state alone.
5. Every state transition is validated server-side against the transition table in §8.2.
6. AI may recommend; it may never execute a transition (§11.5).
7. Admin approval can never bypass verification, limits, or state-machine rules.
8. A RELEASED or SETTLED transaction can never return to PROTECTED.

9. Refunds and disputes are separate post-settlement workflows, never a mutation of the original transaction.
10. Every user, system, scheduler, and admin action is audited, append-only.
-

16. Non-Functional Requirements

Category	Requirement
Performance	LOW-risk settlement must feel instant (<1s perceived latency end-to-end for the confirm action).
Reliability	Scheduler must be idempotent and safe to run from multiple instances (see §8.4).
Data integrity	All monetary values <code>NUMBER(18,2)</code> / <code>BigDecimal</code> ; ledger append-only; DB-level constraints, not just app validation.
Security	JWT auth, Spring Security RBAC, BCrypt password hashing, account lockout after repeated failed logins, OTP never logged in plaintext, device/location data collected with clear consent.
Auditability	Every state-changing action produces an immutable audit row with actor, action, target, old/new values, timestamp, and (for admin actions) a reason.
Accessibility	WCAG 2.1 AA minimum for all customer-facing screens — color is never the sole indicator of risk state (pair with icon/text).
PWA	Installable manifest, service worker for app-shell caching, works offline for read-only history, responsive 360px–1440px+, push-notification opt-in.
Explainability	Every risk decision has a stored, retrievable plain-language explanation (§7.4).

17. Tech Stack

Layer	Technology	Notes
Backend framework	Spring Boot 3.x (Java 17)	Spring MVC on the Servlet API — satisfies a "servlets" requiremer

Layer	Technology	Notes
Security	Spring Security + JWT	Role-based access (CUSTOMER/MAKER/CHECKER/RISK_OFFICER/SYSTEM_ADMIN)
Persistence	Spring Data JPA (Hibernate) + Oracle DB	<code>@GeneratedValue(strategy = GenerationType.SEQUENCE)</code> Oracle sequences
Scheduling	Spring <code>@Scheduled</code>	Periodic scan of expired <code>PROTECTED</code> rows; server-authoritative t (§11.4)
Real-time	Spring WebSocket (STOMP)	Pushes countdown/status so the UI doesn't poll
Frontend	Modern PWA (component framework of choice — e.g., React/Vue + Vite, service worker, Web App Manifest)	Deviates from the source Blueprint's Oracle JET recommendation explicit PWA brief — see §12.1 note
Build tools	Maven (backend); standard PWA build tooling (frontend)	
Testing	JUnit 5 + Mockito	Especially the risk-scoring function (pure logic) and state-machine boundary/race conditions
Deployment	Spring Boot embedded Tomcat; Oracle JDBC driver (<code>ojdbc11</code>)	

18. Problem → Feature Traceability Map

Problem	SafePay feature
Wrong beneficiary	Beneficiary confirmation screen, trust score, protection timer
Wrong amount	Amount confirmation, anomaly rule, custom transaction limit
Duplicate payment	Idempotency key, duplicate detector
Scam pressure / social engineering	High-risk warning copy, timer, extra OTP verification
New beneficiary risk	Beneficiary-age rule + cooling period
Unknown device/location	Device/location risk rule + hard hold
Corporate fraud / internal error	Maker-Checker + approval limits
Silent settlement failures	transaction_exception table + admin visibility
Post-settlement mistake discovery	Dispute workflow (separate from pre-settlement protection)

19. Edge Cases — Consolidated Checklist

(Every item below must have a specific, testable handling behavior before the feature is considered done.)

- Password hashing + account lockout after repeated failed logins.
- Duplicate UPI ID / duplicate beneficiary detection.
- Invalid amount (zero, negative, >2 decimal places) rejected before transaction creation.
- Insufficient simulated balance.
- Risk rules unavailable → default to the safer (more protective) action, never LOW.
- Boundary risk scores (exactly at a tier threshold) — classify deterministically, no ambiguity.
- Server/app restart during an active protection window.
- OTP expired, OTP failed, max OTP attempts reached.
- Cancel attempted at the exact millisecond the timer expires (§8.3).
- Multiple scheduler instances racing to release the same transaction.
- Never expose internal risk-rule codes or OTP values in any customer-facing surface.
- Never phrase a warning as an accusation against the beneficiary.

- Notification delivery failure must not block the payment.
- User cannot set a safety preference that bypasses a mandatory bank-level rule.
- Same beneficiary, same amount submitted twice within a short window → duplicate-payment prompt, exact retry blocked by idempotency key.
- A maker cannot approve their own corporate payment.
- Approval-limit changes mid-review must not retroactively invalidate an in-flight approval silently — must be logged.
- Settlement failure must never be marked SETTLED; must log to `transaction_exception` and either retry (same idempotency key) or move to FAILED.
- Admin cannot view a user's password, cannot silently raise limits, cannot edit past payment history, cannot delete audit events.
- A refund after SETTLED is always a **new** transaction, never a mutation of the original.

20. Success Metrics (KPIs)

Metric	Why it matters
% of LOW-risk payments settled in <1s	Validates "fast when safe"
Cancellation rate during protection windows	Direct evidence of mistakes/scams caught
False-positive rate (low-risk payments unnecessarily delayed)	User-trust erosion signal
False-negative rate (risky payments waved through)	Core efficacy signal
Average protection-window duration by tier	Policy-tuning signal
Dispute rate on SETTLED transactions	Should trend down as protection catches more issues pre-settlement
Checker approval turnaround time (corporate)	Operational efficiency

21. Known Limitations (must be stated in any external-facing doc, per source research)

1. SafePay cannot delay a real UPI/IMPS/RTGS settlement today — a real deployment would sit at the bank's own initiation layer (before it calls the NPCI/RTGS API), not intercept money already in flight.
 2. Friction vs. genuine urgency trade-off — any override path for real emergencies is itself a new attack surface a coached scam victim could exploit.
 3. Risk-engine calibration needs real transaction data; this is a cold-start problem for any new deployment.
 4. Scammer adaptation — a sufficiently coached victim can be walked through the protection screen; SafePay raises the bar, it doesn't eliminate human-trust manipulation as a vulnerability.
 5. Real deployment requires NPCI/RBI cooperation, security certification (VAPT), and likely a regulatory sandbox process.
 6. SafePay does not address the receiving end (money-mule accounts) — that is a separate KYC/AML problem.
-

22. Recommended Build Order

1. Data model + JPA entities + Oracle schema (DDL, sequences, constraints) + seed data.
 2. Auth (register/login/JWT) + RBAC.
 3. Beneficiary CRUD with cooling-off metadata.
 4. Transaction initiate + deterministic risk engine (pure logic — unit-test first, before any UI).
 5. State machine + idempotency + optimistic locking + scheduled expiry job.
 6. Protection window UI + WebSocket countdown push.
 7. Undo/cancel flow with race-condition handling.
 8. OTP step-up verification flow.
 9. Transaction history + plain-language risk explanations.
 10. Basic Admin Dashboard + Risk Review Queue + Audit Viewer.
 11. Notifications (in-app first, simulated email/SMS second).
 12. **Phase 2:** Maker-Checker, user safety preferences, admin-configurable policy, disputes, reporting views, PWA installability/offline/push.
 13. **Future:** Restricted AI recommendation adapter (§11.5), once the deterministic MVP is stable and proven.
-

23. Best Demonstration Scenario (for stakeholder/investor demo)

1. User adds a new beneficiary.
2. User creates a ₹4,50,000 payment from an unfamiliar device.
3. System scores it VERY_HIGH; reasons shown: new beneficiary, large amount, unknown device.
4. System requests OTP, then starts a 60-second protection window.
5. User (realizing it was a scam) clicks **Undo**.
6. Transaction becomes CANCELLED.
7. Dashboard confirms no settlement occurred; Admin Audit Viewer shows the complete decision timeline end-to-end.

24. Glossary

Term	Meaning in SafePay
Authorization	User confirms payment intent (simulated PIN/OTP); not yet settled
Protection window	Timed period in <code>PROTECTED</code> state during which the user may cancel
Cancellation	Stops a transaction before settlement (final)
Release	System permits a protected transaction to proceed, after timer expiry or verification
Settlement	Final simulated debit/credit ledger entry is posted
Refund	A new, separate payment returning money after settlement
Dispute/Chargeback	Formal post-settlement complaint process; does not guarantee return of funds
Idempotency key	Client-supplied token ensuring a retried request is not double-processed
Ledger entry	Immutable, append-only double-entry posting

End of PRD. This document should be re-versioned (v1.1, v1.2...) as policy thresholds, screen lists, or scope tiers change — never silently edited in place once shared with a build tool.